ARTIFICIAL AND COMPUTATIONAL INTELLIGENCE · AIMLCZG557 · COMPANION READER

Agents, PEAS & Task Environments

What is an agent? How do we describe what it must do?

Session 1 · Read alongside the lecture slides · Every slide example worked out in full

How to use this companion. The slides give you the formulas; this reader gives you the *why*. Every slide concept is expanded into a full, plain-English explanation. Every slide example is worked out step by step. Five colour-coded boxes guide you. **Blue** "The intuition" states the core idea in one breath. **Amber** "Everyday picture" ties it to real life before any symbols. **Green** "Worked example" solves a problem with real numbers and no skipped steps. **Red** "Watch out" flags the common mistake. **Purple** "Key takeaway" is the one sentence to remember. Read it in order alongside the slides.

1 What this session covers

Session 1 of ACI sets the stage for the whole course. It answers four big questions.

1. What exactly is *artificial intelligence*? Four perspectives.
2. What is an *agent* and what does it do?
3. How do we describe an agent's task? The *PEAS* framework.
4. What kinds of *environments* can an agent live in? Six properties.

The textbook is *Artificial Intelligence: A Modern Approach*, 4th edition, by Stuart Russell and Peter Norvig (referred to as T1 in the course). Section 1.1 covers perspectives on AI; Chapter 2 covers agents and environments.

2 Four perspectives on artificial intelligence

When researchers try to define AI, they split on two questions. First: should a machine *think* like a human, or should it reason *correctly*? Second: should it *behave* like a human, or should it act *rationally*? Crossing those two choices gives four cells.

☞ The intuition

Think of it as a two-by-two table. One axis is *human* vs. *rational*. The other is *thought* vs. *behaviour*. Most of this course lives in cell four: acting rationally.

	Human performance	Rational performance
Thought / Reasoning	(1) Thinking humanly	(3) Thinking rationally
Acting / Behaviour	(2) Acting humanly	(4) Acting rationally

(1) Thinking humanly — the cognitive modelling approach. The goal is to understand and replicate how humans think. We study human cognition through introspection, psychological experiments, and brain imaging. Deep Blue beat Garry Kasparov at chess, but it does not think like a human. This approach matters for intelligent tutoring systems and human–computer interaction, where human-like reasoning is important.

(2) Acting humanly — the Turing Test approach. Alan Turing proposed a test in 1950. An interrogator exchanges text messages with two hidden parties: a human and a computer. If the interrogator cannot tell which is which, the computer passes. To pass, the machine needs natural language processing (NLP), knowledge representation, automated reasoning, and machine learning.

(3) Thinking rationally — the laws-of-thought approach. Aristotle formalised logic in the third century BC. He introduced **syllogisms** — argument structures that guarantee a correct conclusion given true premises. The classic example: "All humans are mortal; Socrates is human; so Socrates is mortal." The field of formal logic tried to codify all rational thinking. Its problem: not every intelligent action is the result of logical deliberation, and even a few hundred facts can exhaust a computer.

(4) Acting rationally — the rational agent approach. This is the approach this course takes. A **rational agent** acts to achieve the best expected outcome. It does not have to think exactly like a human. It just has to make good decisions.

★ Everyday picture

Aeronautical engineers do not build planes that flap their wings like birds. They find the best way to fly. AI researchers do the same: they build the best decision-maker, not a human imitation.

Where this shows up in ML. All modern ML systems are designed as rational agents. They learn a policy that maximises a reward or loss objective. None of them is required to think the way a human does.

✓ Key takeaway

The four perspectives: (1) thinking humanly, (2) acting humanly, (3) thinking rationally, (4) acting rationally. This course focuses on (4).

3 Agents and environments

Everything in this course is built around one idea.

☞ The intuition

An **agent** is anything that perceives its environment through **sensors** and acts on that environment through **actuators**. The agent reads its sensors, decides what to do, and sends a command to its actuators. That is the complete loop.

★ **Everyday picture**

You are an agent. Your sensors are your eyes, ears, and other senses. Your actuators are your hands, legs, and voice. You perceive the world and act on it. A robot vacuum cleaner is also an agent. Its sensors are a dirt detector and a location sensor. Its actuators are its wheels and its suction motor.

Three kinds of agent appear in the slides.

- **Human agent.** Sensors: eyes, ears, skin, nose. Actuators: hands, legs, mouth.
- **Robotic agent.** Sensors: cameras, infrared range finders. Actuators: various motors.
- **Software (AI) agent.** Sensors: text input, data streams. Actuators: text output, API calls, control signals.

3.1 The agent function and agent program

At each moment the agent has seen a sequence of percepts. We call that the **percept history**. The **agent function** maps any percept history to an action:

$$f : \mathcal{P}^* \to \mathcal{A}$$

Here $\mathcal{P}^*$ (read: "the set of all percept histories") is every possible sequence of percepts the agent could have seen. $\mathcal{A}$ is the set of possible actions. So f is the complete specification of what the agent would do in any situation.

The **agent program** is the actual code that runs on a physical machine and produces f. The short equation the slides give is:

$$\text{agent} = \text{architecture} + \text{program}$$

The architecture is the hardware (CPU, sensors, actuators). The program is the software that makes decisions.

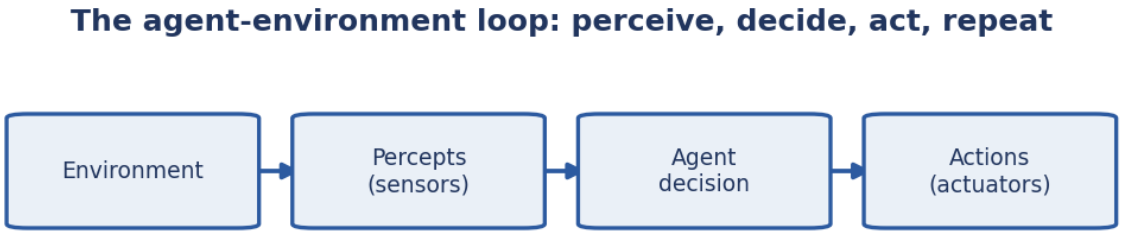

Figure 1: The agent–environment loop. The agent reads percepts from the environment via sensors and sends actions back via actuators. One cycle repeats every time step.

Where this shows up in ML. In reinforcement learning (RL), the agent function f is called the *policy*, usually written π. The policy maps an observation o_t (or history) to an action a_t. Training an RL agent means finding the policy π that maximises expected cumulative reward.

✓ Key takeaway

Agent = architecture + program. The agent function $f : \mathcal{P}^* \to \mathcal{A}$ maps the full percept history to an action.

4 Rational agents

Knowing what an agent *is* does not yet say what a *good* agent does.

☞ The intuition

A rational agent does the *right thing*. The right thing is the action expected to score best on the **performance measure** — the scoring rule that tells us what counts as success.

★ Everyday picture

Think of a student taking an exam. The performance measure is the final mark. A rational student picks answers that maximise the expected mark given the time and knowledge available. They do not need to know every answer; they just need to make the best use of what they know.

4.1 Performance measure

A **performance measure** is an objective criterion for success. It is defined from the *outside* — by the designer who wants certain outcomes — not from the agent's internal preferences.

Example: vacuum cleaner. The performance measure could be any of these:

- Amount of dirt cleaned up.
- Amount of electricity used (we want this small).
- Amount of noise generated (we want this small).
- Time taken to finish.

A designer must choose and combine these carefully. An agent that scores well on one might score poorly on another.

4.2 The ideal rational agent

The slides give this definition.

> "For each possible percept sequence, a rational agent does whatever action is expected to maximise its performance measure. It acts on evidence perceived *so far* and on built-in knowledge."

Three words in this definition deserve attention.

- **Expected.** The agent may face uncertainty. It picks the action with the best *expected* score, not a guaranteed one.
- **So far.** The agent acts on what it has seen up to now. It is not omniscient — it cannot see the future.

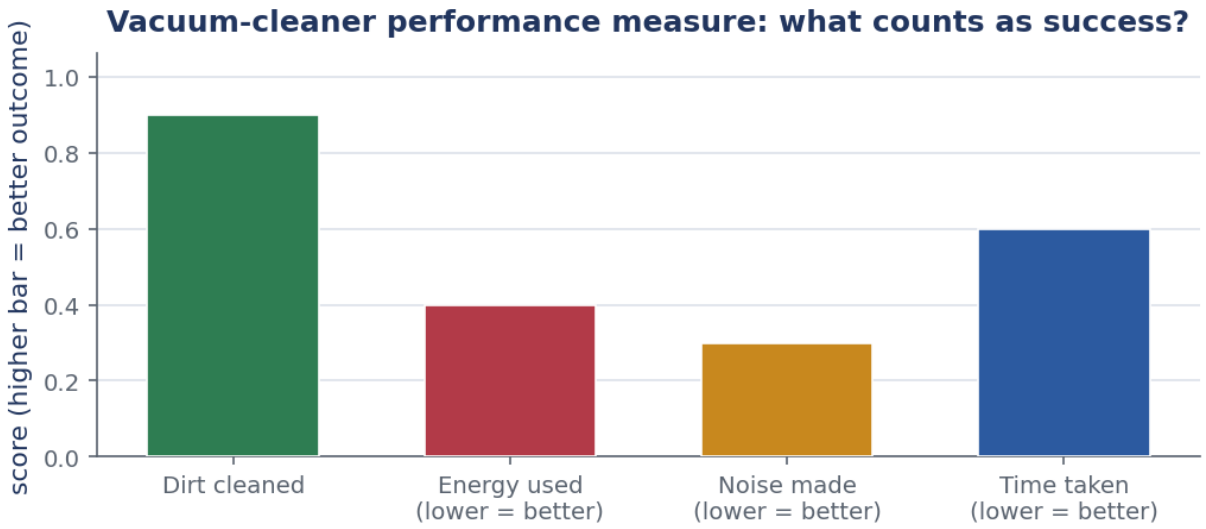

Figure 2: Four components of a vacuum-cleaner performance measure. High bars are good for dirt cleaned; low bars are good for energy, noise, and time. The designer must decide how to weight them — the agent then tries to maximise the combined score.

- **Built-in knowledge.** The agent may have prior knowledge built in by its designer (e.g. a map, rules of a game).

4.3 Rationality is not omniscience

Omniscience means knowing everything, including the future outcome of every action. Rationality does not require omniscience. A rational agent makes the best decision *given its current information*. If the outcome turns out bad, that does not make the agent irrational — it may simply have been unlucky.

An agent is **autonomous** if its behaviour depends on its own experience rather than only on the prior knowledge built in by its designer. An autonomous agent can learn and adapt.

4.4 Bounded rationality

Perfect rationality requires searching over all possible actions and all possible futures. That is computationally impossible in most real settings. **Bounded rationality** is the practical version: do the best you can given limited time and limited computational resources. The agent optimises its performance measure *subject to* a computational budget.

✎ Worked example: Vacuum cleaner rationality

The vacuum cleaner is in room A, which is dirty. Room B's state is unknown.

Step 1. **State the setup.** Percept: [A, Dirty]. Actions available: *Left*, *Right*, *Suck*, *NoOp*.

Step 2. **Identify the rational action.** Room A is dirty. Sucking cleans it right now. Moving to B first would leave A dirty longer. So the rational action is **Suck**.

Step 3. **Check the next step.** After Suck, the new percept might be [A, Clean]. Now the agent does not know whether B is dirty. A rational agent explores — it moves Right to check.

Step 4. **Sense-check.** This matches the slide's percept–action table: [A, Dirty] → Suck

and [A, Clean] → Right.

The key insight: rationality means acting on *available* evidence, not on perfect knowledge of the world.

Where this shows up in ML. Every supervised and reinforcement learning algorithm is a search for a rational policy. In RL, the Bellman equation formalises expected future reward. That is exactly "expected performance on the basis of evidence so far" from the definition above.

✓ Key takeaway

A rational agent maximises *expected* performance on the basis of evidence so far. Rationality ≠ omniscience. Bounded rationality is the real-world version: best action given limited computation.

5 PEAS: specifying the task environment

Before we can design an agent we need to describe its task precisely. The standard tool is **PEAS**: **P**erformance measure, **E**nvironment, **A**ctuators, **S**ensors.

☞ The intuition

PEAS is a job description for the agent. *Performance* says what success looks like. *Environment* says where the agent lives. *Actuators* say what it can do. *Sensors* say what it can see. Write all four before writing a single line of agent code.

★ Everyday picture

Imagine hiring a taxi driver. Before the interview you write a job spec. Success means safe, fast, legal, comfortable trips — that is P. The workplace is city roads with other traffic and pedestrians — that is E. The tools are a steering wheel, accelerator, brake, and horn — that is A. The information comes from eyes, GPS, and a speedometer — that is S. PEAS is that job spec, but for an AI system.

✎ Worked example: PEAS for the automated taxi driver (slide example)

The slide lists this agent explicitly. We work through all four letters.

Step 1. **Performance measure.** Safe trip; arrive at the correct destination; obey traffic laws; travel quickly; ride is comfortable; maximise profit.

Step 2. **Environment.** Roads, other vehicles, pedestrians, weather, traffic signals, customers, maps, GPS data.

Step 3. **Actuators.** Steering wheel, accelerator pedal, brake pedal, horn, turn signals.

Step 4. **Sensors.** Cameras (front, rear, side), sonar or LIDAR for distance, GPS receiver, speedometer, odometer, engine sensors, keyboard (for passenger input).

Full PEAS in one line: **P** safe/fast/legal/comfortable trip; **E** roads, traffic, pedestrians, weather; **A** steering, accelerator, brake, horn, signals; **S** cameras, LIDAR, GPS, speedometer, odometer.

✎ **Worked example: PEAS for the vacuum cleaner (slide example)**

The vacuum lives in a two-room world with rooms A and B.

Step 1. **Performance measure.** Maximise cleanliness (number of clean squares over time). Minimise energy consumption.

Step 2. **Environment.** Two rooms, A and B. Each room is either Clean or Dirty. The agent starts in one of them.

Step 3. **Actuators.** *Left* (move to A), *Right* (move to B), *Suck* (clean current room), *NoOp* (do nothing).

Step 4. **Sensors.** Location sensor: which room am I in? Dirt sensor: is the current room Dirty or Clean?

Full PEAS: **P** clean squares, energy; **E** 2-room grid, each dirty or clean; **A** Left, Right, Suck, NoOp; **S** location, dirt detector.

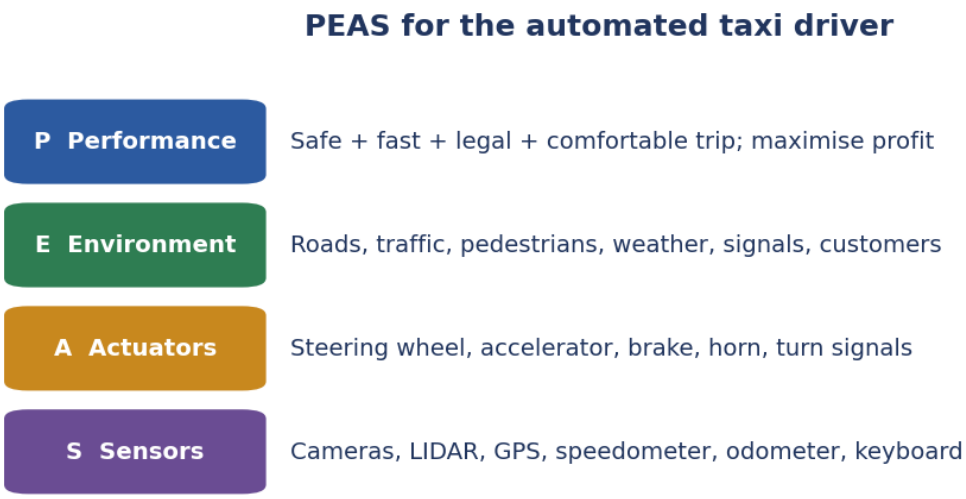

Figure 3: PEAS for the automated taxi. Each letter of PEAS is a separate design decision. Get the performance measure wrong and the agent optimises the wrong thing.

Where this shows up in ML. Every RL framework starts with a PEAS-like specification. In OpenAI Gym / Gymnasium: *performance measure* is the reward function, *environment* is the `env` object, *actuators* are the action space, *sensors* are the observation space.

✗ **Watch out**

The performance measure must be defined from *outside* the agent — by the designer, not by the agent itself. If you let the agent define its own performance measure it will find clever ways to score well without doing what you actually wanted. This is the *reward hacking* problem in RL.

✓ **Key takeaway**

PEAS = Performance, Environment, Actuators, Sensors. Write all four before building any agent. The performance measure is set by the designer; the agent tries to maximise it.

6 Properties of task environments

Different environments are harder or easier for an agent to handle. We describe every environment along six independent dimensions.

☞ The intuition

These six pairs are a checklist. Read the problem statement, tick each pair, and you have a complete picture of how hard the design problem is. Harder environments demand more sophisticated agents.

Dimension	One end	Other end
Observability	Fully observable	Partially observable
Number of agents	Single-agent	Multi-agent (competitive or cooperative)
Outcome of actions	Deterministic	Stochastic
Episode structure	Episodic	Sequential
Change over time	Static	Dynamic
State / action space	Discrete	Continuous

Fully observable means the agent's sensors give it the complete state at every step. Chess: you can see every piece — fully observable. **Partially observable** means some state is hidden. Poker: you cannot see your opponents' cards — partially observable.

Deterministic means the next state is fully determined by the current state and the chosen action. No randomness at all. **Stochastic** means there is uncertainty: the outcome may vary even if you take the same action in the same state. Rolling dice, slippery floors, sensor noise.

Episodic means each task episode is independent. What happened in episode 1 has no effect on episode 2. Sorting an email inbox: each email is a fresh decision. **Sequential** means past actions affect future states. Chess: every move changes what moves are legal later.

Static means the environment does not change while the agent is thinking. A crossword puzzle waits for you. **Dynamic** means it can change at any time — even mid-deliberation. Driving: other cars keep moving while you think. A **semi-dynamic** environment does not change, but the agent's performance score changes with time (e.g. a timed exam).

Discrete means a finite (or countably infinite) set of states and actions. Chess: fixed number of board positions and legal moves. **Continuous** means states or actions take real-valued (numerical) values. Self-driving car: steering angle, speed, and position are all continuous.

✎ Worked example: Classify the vacuum-cleaner world

The two-room vacuum world from the slides. We justify each property.

Step 1. **Observability.** The agent sees its location and the dirt status of its current room. But it cannot see room B while standing in A. **Partially observable.**

Step 2. **Number of agents.** Only one vacuum. **Single-agent.**

Step 3. **Determinism.** In the basic version, sucking always cleans. Moving always changes location. **Deterministic.**

Step 4. **Episode structure.** Does what happened last time matter? If rooms can get dirty again after cleaning, past actions matter. **Sequential.**

Step 5. **Change over time.** In the basic version the rooms do not get dirtier while the

agent thinks (no time pressure changes state). **Static.**

Step 6. **State space.** Two rooms, two dirt states each = 4 world states; 4 actions. **Discrete.**

Result: Partially observable, Single-agent, Deterministic, Sequential, Static, Discrete.

✎ **Worked example: Classify online chess against a human**

The slides also ask about chess. We justify each property.

Step 1. **Observability.** Both players see the full board. **Fully observable.**

Step 2. **Number of agents.** Two players, each trying to win. **Multi-agent, competitive.**

Step 3. **Determinism.** Pieces move to exactly the stated square. No dice. **Deterministic.**

Step 4. **Episode structure.** Every past move restricts future legal moves. **Sequential.**

Step 5. **Change over time.** The board does not change while you are thinking. **Static.**

Step 6. **State space.** Finite number of squares and piece types. **Discrete.**

Result: Fully observable, Multi-agent competitive, Deterministic, Sequential, Static, Discrete.

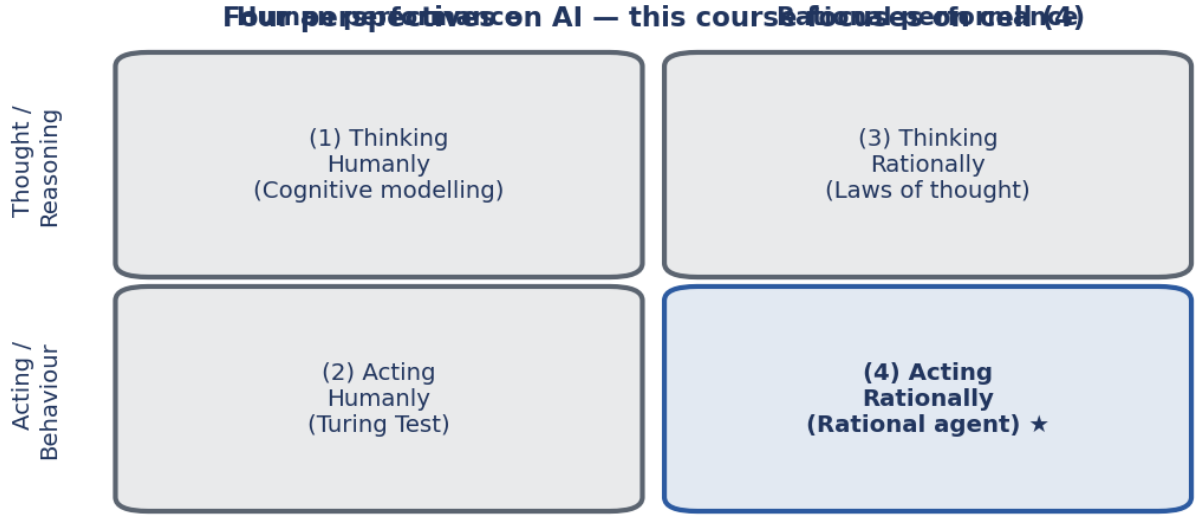

Figure 4: Four perspectives on AI and the six environment dimensions. The cell labelled "acting rationally" is where this course lives; harder environments (bottom-right) need more capable agents.

Where this shows up in ML. RL researchers use these six dimensions to describe benchmark environments. Atari games: fully observable, single-agent, deterministic, episodic, dynamic, discrete — a good starting point. Real-world robotics: partially observable, single-agent, stochastic, sequential, dynamic, continuous — much harder. Knowing the properties tells you which algorithms will work.

✗ Watch out

Two common mix-ups at the exam.
First: *static* vs. *deterministic*. Static means the environment does not change while the agent is thinking. Deterministic means there is no randomness in the outcome of actions. An environment can be static *and* stochastic. A card-shuffling puzzle is static (nothing moves while you think) but stochastic (you draw a random card).
Second: *multi-agent* does not always mean *competitive*. Two robots cooperating to carry a sofa are multi-agent but cooperative. Two chess players are multi-agent and competitive.

✓ Key takeaway

Six dimensions: (1) fully vs. partially observable; (2) single vs. multi-agent; (3) deterministic vs. stochastic. Also: (4) episodic vs. sequential; (5) static vs. dynamic; (6) discrete vs. continuous. Harder environments demand more sophisticated agents.

7 The agent–environment percept table

The slide shows the vacuum cleaner's full percept–action mapping. This table is the agent function f written out explicitly.

☞ The intuition

Every row is one possible percept (or percept history). The right column gives the action the agent takes. An agent that always acts rationally fills in this table optimally. For simple environments we can write the whole table down. For complex ones we need a program to approximate it.

Percept sequence (what the agent has seen so far)	Action
[A, Clean]	Right
[A, Dirty]	Suck
[B, Clean]	Left
[B, Dirty]	Suck
[A, Clean], [A, Clean]	Right
[A, Clean], [A, Dirty]	Suck
⋮	⋮
[A, Clean], [A, Clean], [A, Clean]	Right
[A, Clean], [A, Clean], [A, Dirty]	Suck
⋮	⋮

Notice two things. First, the table grows without bound: there is one row for every possible percept history, and histories can be arbitrarily long. Second, the last action is always based on the *whole history*, not just the latest percept. A simple reflex agent ignores history; a model-based agent summarises it.

Where this shows up in ML. A look-up table agent is the simplest possible policy. In RL, the Q-table in tabular Q-learning is exactly this: a table mapping (state, action) pairs to expected rewards. For large state spaces, a neural network compresses the table into a parameterised function.

Vacuum-cleaner agent: percept sequence → action (the agent function)

Percept sequence	Action
[A, Clean]	Right
[A, Dirty]	Suck
[B, Clean]	Left
[B, Dirty]	Suck
[A,C],[A,C]	Right
[A,C],[A,D]	Suck

Figure 5: The vacuum-cleaner percept–action table. Each row is one percept history. The agent always picks the action in the right column. The table is the agent function written out in full.

✓ Key takeaway

The percept–action table *is* the agent function. It maps every possible percept history to an action. Real agents approximate this table because it is too large to store.

8 A brief history and foundations of AI

Understanding where AI came from helps you understand why the rational agent view won.

Key milestones.

- 1950: Alan Turing proposes the Turing Test.
- 1956: John McCarthy coins the term "Artificial Intelligence" at the Dartmouth Conference.
- 1957–1973: Early optimism. First programs, first robots. First "AI Winter" follows (funding cuts, overpromising).
- 1980–1987: Expert systems boom. Second "AI Winter" follows.
- 1997: IBM Deep Blue defeats world chess champion Kasparov.
- 2006: Deep Belief Networks (Hinton et al.).
- 2012: AlexNet wins ImageNet; deep learning era begins.
- 2017: Transformer architecture ("Attention Is All You Need").
- 2022: ChatGPT launches; large language models go mainstream.
- 2025: DeepSeek R1 and rapid advances in agentic AI.

What changed in the last decade? Three forces converged: *data* (internet scale), *compute* (GPUs, TPUs), and *algorithms* (deep learning, transformers). Each alone was not enough; all three together unlocked modern AI.

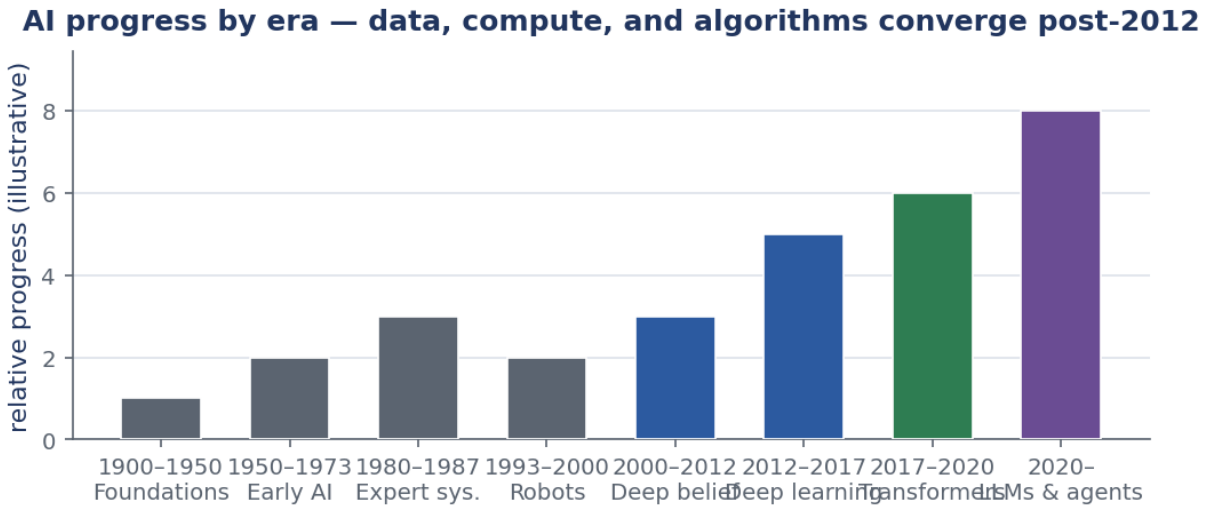

Figure 6: Key milestones in AI history. Two "winters" (funding cuts) followed periods of overpromising. Modern AI took off when data, compute, and deep-learning algorithms converged.

Foundations of AI. AI draws on many fields. These include philosophy (logic, rationality), mathematics (probability, computation), and economics (decision theory, game theory). Neuroscience, psychology, computer engineering, and linguistics all contribute too.

Where this shows up in ML. Transformer-based large language models are the direct result of that 2017 milestone. Understanding the history helps you see why the field is structured the way it is — and where the next big shift might come.

✓ **Key takeaway**

AI has gone through cycles of optimism and winters. Modern AI is powered by the convergence of data, compute, and algorithms. The rational agent framework is the unifying idea across all of it.

9 AI Agents vs. Agentic AI

The slides draw a distinction that students often miss.

☞ **The intuition**

An **AI agent** solves a specific task using learned or programmed rules. **Agentic AI** is a network of agents with high autonomy: they plan, delegate sub-tasks, adapt, and coordinate without a human directing every step. The key difference is the *level of autonomy.*

Example: customer support. An AI agent answers frequently-asked questions using a knowledge base. It has fixed inputs and outputs. An agentic AI system would also escalate complaints, analyse customer sentiment, adapt its scripts, and coordinate with backend systems to resolve problems — all autonomously. The orchestration and planning are themselves learned, not hard-coded.

This course focuses on rational agents: we study the principles (search, logic, probability) that underlie all AI systems. We do not go into the specifics of modern agentic pipelines (MCP, tool-calling, LLM orchestration).

✓ **Key takeaway**

An AI agent solves a fixed task by perceiving and acting. Agentic AI is a higher-autonomy system where planning and coordination are also automated. This course focuses on the rational-agent foundation.

Glossary & symbols

Key terms.

agent anything that perceives its environment and acts on it.

percept a single reading from the agent's sensors at one time step.

actuator the part of the agent that produces actions (hands, wheels, API calls).

agent function the mapping $f : \mathcal{P}^* \to \mathcal{A}$ from percept history to action.

agent program the code that implements the agent function on real hardware.

performance measure the scoring rule that says what counts as success.

rational agent an agent that picks the action expected to maximise its performance measure.

omniscience knowing the actual outcome of every action — not required for rationality.

autonomy the degree to which an agent's behaviour is driven by its own experience, not built-in knowledge.

bounded rationality doing the best you can given limited computation and time.

PEAS Performance, Environment, Actuators, Sensors — the four-part task description.

fully observable the sensors reveal the complete state of the environment.

partially observable some state is hidden from the agent's sensors.

deterministic the next state is fully determined by the current state and action.

stochastic the outcome of an action has some randomness.

episodic each decision episode is independent; past episodes do not affect future ones.

sequential past actions affect future states.

static the environment does not change while the agent deliberates.

dynamic the environment can change at any time, even mid-deliberation.

discrete states and actions come from a finite (or countable) set.

continuous states or actions take real-valued numbers.

Turing Test a test of machine intelligence proposed by Alan Turing (1950). A machine passes if an interrogator cannot tell it apart from a human in text conversation.

syllogism a logical argument form with two premises and a conclusion; introduced by Aristotle.

Symbols at a glance.

Symbol	Meaning
$\mathcal{P}$	the set of possible percepts
$\mathcal{P}^*$	the set of all finite percept histories (sequences of zero or more percepts)
$\mathcal{A}$	the set of possible actions
f	the agent function; maps a percept history to an action
π	the policy in reinforcement learning (same concept as f)